UNIVERSIDAD DE LOS LLANOS

Facultad de Ciencias Básicas e Ingeniería — Programa de Ingeniería de Sistemas

Asignatura: Procesos Estocásticos (Código 603504) — 5.º Semestre

ESPECIFICACIÓN OFICIAL DEL PROYECTO FINAL

Reconocedor y Clasificador de Caracteres en Tiempo Real 2D/3D mediante Cadenas Ocultas de Markov (HMM)

Docente: Yerson Porras (yerson.porras@unillanos.edu.co)
Modalidad: Grupal (Máximo 4 estudiantes)
Ponderación: 90% del Tercer Corte
Evaluación: Nota Conjunta y Defensa In

1. Descripción General y Propósito del Proyecto

El proyecto final de la asignatura **Procesos Estocásticos** tiene como objetivo integrar la teoría estocástica avanzada de **Cadenas Ocultas de Markov (Hidden Markov Models - HMM)** con el desarrollo de software interactivo en tiempo real y los principios formales de arquitectura de software.

Los equipos de trabajo deberán diseñar, desarrollar e implementar desde cero una aplicación de escritorio con interfaz gráfica de usuario (GUI) capaz de capturar, procesar, clasificar y autocompletar dinámicamente trazos de caracteres realizados a mano alzada tanto en un **lienzo bidimensional (2D)** como en un **lienzo tridimensional esférico (3D)**.

La piedra angular del proyecto radica en la **implementación matemática nativa de los algoritmos de Cadenas Ocultas de Markov** (sin utilizar librerías externas para la lógica estocástica), aplicando la teoría aprendida para resolver los problemas de verosimilitud, decodificación y proyección espacial de trayectorias futuras.

2. Requerimientos Funcionales de la Interfaz Gráfica (GUI)

La aplicación debe presentar un diseño visual limpio, moderno y minimalista, estructurado en tres ventanas o secciones principales:

2.1. Ventana A: Menú Principal de Inicio

- **Módulo de Entrada:** Pantalla de bienvenida institucional que permite seleccionar el modo de trabajo:
  1. **Modo 2D:** Reconocimiento y proyección de trazos en plano cartesiano bidimensional.
  2. **Modo 3D:** Reconocimiento y proyección de trazos sobre la superficie de una esfera tridimensional.
- **Navegación:** Botones intuitivos para alternar entre modos o regresar al menú principal.

2.2. Ventana B: Reconocedor y Visualizador 2D

La interfaz se divide en dos secciones principales:

1. **Sección Izquierda (Lienzo 2D de Trazado y Proyección):**
   - **Captura en Tiempo Real:** Área interactiva de dibujo mediante ratón o dispositivo táctil.

- **Proyección Dinámica de Trayectoria (Autocompletado Espacial):** A medida que el usuario realiza el trazo (paso a paso en el tiempo t), la GUI proyecta de forma transparente o punteada la trayectoria futura estimada. Esta proyección corresponde al carácter que presente la mayor probabilidad a posteriori en el instante actual. Si el trazo evoluciona y la predicción cambia a otra letra, la proyección proyectada debe actualizarse en tiempo real.
- **Controles:** Botones para "Limpiar Lienzo" y "Reiniciar Captura".

2. **Sección Derecha (Panel de Clasificación y Probabilidades):**

- **Listado Ordenado Descendente:** Muestra en tiempo real las letras clasificadas según su probabilidad estimada $P(\text{Carácter} \mid O_{1:t})$.
- **Representación Visual:** Cada elemento exhibe la letra, la probabilidad acumulada (porcentaje o valor $[0.0, 1.0]$) y un icono/pequeña imagen estática de referencia.

2.3. Ventana C: Reconocedor y Visualizador 3D (Lienzo Esférico)

Mantiene la misma estructura modular que la Ventana B (Lado Izquierdo: Lienzo; Lado Derecho: Listado):

1. **Sección Izquierda (Lienzo 3D Esférico):**

- **Lienzo de Trazado sobre Esfera:** Renderizado 3D de una esfera que actúa como soporte de dibujo.
- **Conversión Angular:** Las coordenadas cartesianas (x, y, z) del trazo sobre la superficie se convierten a **coordenadas esféricas** (θ, ϕ) para medir variaciones angulares y tangenciales.
- **Proyección Dinámica 3D:** Proyección transparente/punteada sobre la superficie esférica.
- **Interacción 3D:** Soporte para rotación, zoom y reorientación de la esfera.

2. **Sección Derecha (Panel de Clasificación 3D):** Mismo listado ordenado descendentemente por probabilidad con iconos de referencia.

3. Requerimientos Matemáticos y Modelado Estocástico (HMM desde Cero)

3.1. Restricción Absoluta sobre Librerías Matemáticas

> **Prohibición Expresa:** Queda **estrictamente prohibido el uso de librerías o frameworks externos** (tales como `hmmlearn`, `scikit-learn`, `PyTorch`, `TensorFlow`, `scipy.stats`) para la formulación, cálculo o resolución de las Cadenas Ocultas de Markov.

- **Implementación Nativa:** Toda la lógica estocástica —incluyendo la estructura del modelo $\lambda = (A, B, \boldsymbol{\pi})$, la evaluación Forward-Backward, la decodificación de Viterbi, la estimación de probabilidades y la proyección futura— debe ser escrita a mano utilizando código nativo (se permite exclusivamente `NumPy` para operaciones algebraicas con arreglos).
- **Demostración Oral:** Los algoritmos deberán ser explicados y demostrados a mano en tablero durante la sustentación oral.

3.2. Formulación del Modelo HMM $\lambda = (A, B, \boldsymbol{\pi})$

Para cada carácter del alfabeto se define un modelo HMM independiente $\lambda_c = (A_c, B_c, \boldsymbol{\pi}_c)$:

1. **Espacio de Estados Ocultos (S):** Fases secuenciales no observables en la construcción del trazo.
2. **Alfabeto de Observaciones (V):** Trazos discretizados a partir de la secuencia continua del usuario.
3. **Matriz de Transición ($A = [a_{ij}]$):** Avance probabilístico entre las fases del trazo.
4. **Matriz de Emisión ($B = [b_j(k)]$):** Probabilidad de emitir una dirección/orientación en cada fase.
5. **Vector Inicial ($\boldsymbol{\pi}$):** Probabilidades de inicio en los estados latentes.

3.3. Conjunto de Caracteres a Clasificar

El sistema debe estar capacitado para reconocer las **27 letras minúsculas del alfabeto español**:

$$\mathcal{C} = \{a, b, c, d, e, f, g, h, i, j, k, l, m, n, \tilde{n}, o, p, q, r, s, t, u, v, w, x, y, z\}$$

(Se excluyen los dígrafos "ch" y "ll", así como las vocales con tildes).

3.4. Extracción Autónoma de Características y Discretización

Es responsabilidad e iniciativa del equipo investigar y proponer un mecanismo de discretización del trazo:

- **Código de Cadena (Chain Code / Cuantización Angular):** Calcular los vectores de desplazamiento $(\Delta x, \Delta y)$ y cuantizarlos en $K = 8$ o $K = 16$ sectores angulares.
- **Coordenadas Esféricas en 3D:** Transformar los puntos de la esfera (x, y, z) a coordenadas angulares (θ, ϕ) y evaluar variaciones $(\Delta\theta, \Delta\phi)$ sobre la superficie esférica.

3.5. Modelo de Proyección de Trayectoria Futura

1. Dada la secuencia parcial $O_{1:t}$, se evalúa la verosimilitud $P(O_{1:t} \mid \lambda_c)$ usando el algoritmo **Forward** ($\alpha_t(i)$).
2. Se identifica el carácter de máxima probabilidad: $\hat{c} = \arg\max_{c \in \mathcal{C}} P(O_{1:t} \mid \lambda_c)$.
3. Utilizando la trayectoria aprendida del modelo $\lambda_{\hat{c}}$, se extraen los símbolos restantes $\{O_{t+1}, \ldots, O_T\}$ y se reconstruyen las coordenadas proyectadas desde la posición del cursor.

4. Requerimientos Técnicos y Arquitectura de Software

4.1. Independización de Modelos y Entrenamiento Previo

- **Sin Modo Entrenamiento en la GUI:** La interfaz no incluye un botón o modo de calibración en tiempo de ejecución.
- **Entrenamiento Offline:** Los parámetros $\lambda_c = (A_c, B_c, \boldsymbol{\pi}_c)$ para los 27 caracteres deben ser entrenados de forma previa mediante scripts independientes (Baum-Welch o máxima verosimilitud).
- **Carga de Modelos:** Los parámetros aprendidos se exportan a archivos estructurados (`.json`, `.npy` o `.pkl`) que la GUI cargará al iniciar.

4.2. Principios de Arquitectura Limpia y POO

El código debe estructurarse modularmente separando capas claras:

1. `HMMEngine.py`: Módulo matemático con la implementación pura de HMM, Forward, Viterbi y proyección.
2. `FeatureExtractor.py`: Módulo de procesamiento geométrico, cuantización vectorial y conversión esférica.
3. `GUI.py` / `Views`: Módulo de interfaz gráfica y eventos de usuario.

4.3. Mini-Biblioteca de Opciones Sugeridas

- **Para GUI y Ventanas:** `PyQt5` / `PySide6`, `CustomTkinter` / `Tkinter`, `Kivy`.
- **Para Lienzo 3D Esférico:** `PyOpenGL` / `OpenGL`, `PyQtGraph` (Módulo 3D), `Matplotlib 3D` (`mplot3d`), `PyGame`.

5. Entregables y Metodología de Evaluación

5.1. Modalidad y Canal de Entrega Institucional

- **Equipos:** Máximo 4 estudiantes (nota grupal y conjunta).
- **Repositorio GitHub:** Código fuente completo, organizado y documentado.
- **Envío de Comprimido (`.zip`):** Envío obligatorio del paquete al correo institucional del docente:

Correo Docente: `yerson.porras@unillanos.edu.co`

5.2. Sustentación Oral Obligatoria

La sustentación presencial se compone de:

1. **Prueba de Trazo en Vivo:** Evaluación de la GUI en 2D y 3D, probabilidades y autocompletado en tiempo real.
2. **Defensa Matemática en Tablero:** Explicación a mano en tablero de las ecuaciones programadas ($\alpha_t(i)$, Viterbi, proyección de trayectoria y modelo HMM), demostrando la implementación nativa.

5.3. Ponderación y Rúbrica de Evaluación (90% del Corte 3)

Nota Final del Proyecto Final = 90% de la calificación total del Tercer Corte

Criterio de Evaluación	**Ponderación**	**% Ponderado**
1. Rigor Matemático e Implementación HMM Nativa (Sin lib)	30 Puntos	27.0%
2. Trazado 2D/3D y Proyección Dinámica de Trayectoria	25 Puntos	22.5%
3. Interfaz Gráfica GUI (Minimalista, Ventanas, Listado Top)	20 Puntos	18.0%
4. Sustentación Oral y Defensa Matemática en Tablero	15 Puntos	13.5%
5. Arquitectura Limpia (POO, Repositorio GitHub y ZIP)	10 Puntos	9.0%
TOTAL DEL PROYECTO FINAL	**100 Puntos**	**90.0% (Corte 3)**